

The Dawn of the Autonomous AI Builder

Participant setup and integration guide: Lyzr × Qdrant × Omi

This document takes you from zero to a working loop: speak into Omi, store the memory in Qdrant, and let a Lyzr agent reason over it. Follow the steps in order, then use the resource list and the submission checklist at the end.

1. Hackathon details

Item	Detail
Organizers	AI House x HiDevs
Tech Stacks	Lyzr × Qdrant × Omi
Prizes	₹20,000 prize pool, developer tier credits, direct ecosystem grant tracks
Submission deadline	13 October 2026, 11:59 PM IST
Winners announced	15 October 2026, 9:00 PM IST
Team size	Solo participants only (1 person per entry)
Workshops	3 Oct and 10 Oct 2026, 8:00 - 10:00 PM IST: "Build a Real-World AI Agent with Lyzr, Omi & Qdrant". The link is in the WhatsApp group.
Resources	API keys, integration guides and starter kits are on the hackathon platform

The timeline was extended because several participants had trouble building agents with Omi. The workshops are hands-on and cover what this guide covers.

Mandatory stack

Every solution must integrate all three:

- **Omi:** Real-time voice capture and ambient input.
- **Qdrant:** Persistent vector memory and semantic retrieval.
- **Lyzr:** Multi - Agent orchestration, reasoning and task execution.

Every project must connect real-time voice input, vector retrieval and orchestrated agent reasoning in one execution loop. Working software, a clean repository and observable agent workflows make up 60% or more of the final score.

Build tracks

Track	What to build
Meeting and lecture intelligence	Voice-to-insight engines with semantic retrieval, automated action-item extraction and Q&A over past meetings.

Track	What to build
Collaborative multi-agent workflows	Voice-triggered pipelines where specialized agents coordinate through a shared vector memory layer.
Privacy-first knowledge systems	Secure, transparent local or semi-local memory with granular, user-controlled visibility.
Accessibility and adaptive assistants	Voice-driven engines that generate structured outputs, forms and guided workflows for high-friction interfaces.
Continuous health and habit tracking	Frictionless voice daily logs with temporal trend detection and time-series semantic retrieval.
Open innovation	Wildcard: devtools, finance, automated research, gaming or autonomous operations.

2. How the three tools connect

Omi cannot talk to Lyzr or Qdrant directly. You write one small backend (FastAPI in this guide) that sits in the middle. The flow is:

1. Omi sends transcripts to your backend through webhooks.
2. Your backend turns the text into a vector and stores it in Qdrant.
3. When you ask a question, the backend searches Qdrant for the most relevant memories.
4. The backend sends those memories plus your question to your Lyzr agent.
5. The Lyzr agent answers or extracts action items, and the backend returns the result.

Why the backend does the retrieval: Lyzr runs in the cloud and cannot reach a Qdrant instance on your laptop. Letting your backend query Qdrant and pass the results to Lyzr works with a local database, a Docker container or Qdrant Cloud, and keeps the memory layer under your control. That matters for the privacy track.

3. Before you start

- Python 3.10 or newer, and pip.
- Docker (recommended for Qdrant). Without Docker, Qdrant's Python client can also run locally from a folder (see step 5).
- A Lyzr Agent Studio account.
- The Omi app on your phone. An Omi device is ideal, but the phone microphone also feeds the app.
- A tunnel that gives your laptop a public HTTPS URL for Omi's webhooks: ngrok or cloudflared.

Install everything once

```
pip install fastapi uvicorn requests qdrant-client fastembed
```

4. Step 1: Create your agent on Lyzr

1. Sign up at studio.lyzr.ai and verify your email. Verification unlocks the visual builder and API keys.
2. Click New Agent. Give it a name and a short description, and pick a default LLM.
3. Paste the system prompt below into the agent's instructions, then test it in the built-in chat.
4. Copy two values and keep them safe: the agent ID and your API key. You will put both in environment variables.

Starter system prompt (adapt it to your track)

```
You are a voice-memory assistant.
Each message contains CONTEXT (snippets retrieved from the user's own
voice memories) and a QUESTION.
- Answer only from the CONTEXT. If the answer is not there, say so.
- When asked, extract action items as a short bulleted list with owners
  and deadlines if they were spoken.
- Keep answers short and cite which snippet you used.
```

Calling the agent from code

Every agent can be called over REST with the chat endpoint. This is the call your backend makes.

lyzr_test.py

```
import os, requests

r = requests.post(
    "https://agent-prod.studio.lyzr.ai/v3/inference/chat/",
    headers={"Content-Type": "application/json",
             "x-api-key": os.environ["LYZR_API_KEY"]},
    json={
        "user_id": "you@example.com",
        "agent_id": os.environ["LYZR_AGENT_ID"],
        "session_id": "test-session-1",
        "message": "CONTEXT: none\nQUESTION: Say hello.",
    },
    timeout=60,
)
print(r.json()["response"])
```

Going multi-agent

The hackathon rewards orchestration. In Studio you can create several specialized agents (for example a summarizer, an action-item extractor and a planner) and put a Manager Agent above them. The manager breaks a request into subtasks and delegates them to worker agents. You can also drive agents from code with the Lyzr ADK (pip install lyzr-adk). Links are in section 10.

Note: Studio menu names change as the product evolves. If a button is not where this guide says, follow the Lyzr quickstart linked in section 10.

5. Step 2: Start your Qdrant vector database

Pick one of the two local options. Neither needs an account or an API key.

Option A: Docker (recommended)

Terminal

```
docker run -p 6333:6333 -v $(pwd)/qdrant_storage:/qdrant/storage qdrant/qdrant:latest
```

Qdrant is now at <http://localhost:6333> and its web dashboard is at <http://localhost:6333/dashboard>. The volume flag keeps your memories after a restart.

Option B: No Docker, embedded local mode

Python

```
from qdrant_client import QdrantClient

client = QdrantClient(path="./qdrant_data") # saved to disk
# client = QdrantClient(":memory:") # throwaway, for quick tests
```

If you prefer a hosted database, Qdrant Cloud works the same way with `QdrantClient(url=..., api_key=...)`.

Embeddings

A vector database stores numbers, so each transcript is first turned into a vector. This guide uses `fastembed`, which runs a small model locally with no API key. The model `BAAI/bge-small-en-v1.5` produces 384-dimension vectors, so your collection must be created with `size=384`. The model downloads once, the first time you run it.

6. Step 3: Get voice input from Omi

Omi sends data to your backend through webhooks. For this project you need two kinds:

Webhook	When it fires	What you receive
Memory creation	After a conversation ends and Omi has processed it.	POST /your-endpoint?uid=USER with the full conversation: transcript segments, a structured summary (title, overview), action items and timestamps.
Real-time transcript	While Omi is actively listening.	POST /your-endpoint?session_id=...&uid=USER with transcript segments: text, speaker, is_user, start, end.

There is also a raw audio stream (PCM16, mono, 16 kHz) if you want to run your own speech recognition. Most projects do not need it.

1. Install the Omi app and pair your device, or use the phone microphone.
2. Open Settings and turn on Developer Mode.
3. Before writing any code, point the webhook fields at a free webhook.site URL and speak. This shows you the exact JSON Omi sends.
4. Later, replace that URL with your tunnel URL from step 4, for example <https://abc123.ngrok-free.app/omi/conversation>.

Important: your endpoint must answer 200 OK quickly and read the uid query parameter. Do slow work after responding or keep it light, otherwise Omi times out.

7. Step 4: Connect Omi, Qdrant and Lyzr

This single file is a complete working loop. It receives Omi transcripts, stores them in Qdrant, and exposes an /ask endpoint that retrieves memories and calls your Lyzr agent.

app.py

```
import os, time, uuid
from typing import Any

import requests
from fastapi import Body, FastAPI
from fastembed import TextEmbedding
from qdrant_client import QdrantClient
from qdrant_client.models import (Distance, FieldCondition, Filter,
                                  MatchValue, PointStruct, VectorParams)

LYZR_API_KEY = os.environ["LYZR_API_KEY"]
LYZR_AGENT_ID = os.environ["LYZR_AGENT_ID"]
COLLECTION = "voice_memory"

# --- Qdrant + embeddings ---
embedder = TextEmbedding("BAAI/bge-small-en-v1.5") # 384 dimensions
qdrant = QdrantClient(url="http://localhost:6333") # or path="./qdrant_data"
if not qdrant.collection_exists(COLLECTION):
    qdrant.create_collection(
        COLLECTION,
        vectors_config=VectorParams(size=384, distance=Distance.COSINE),
    )

def embed(text: str) -> list:
    return next(iter(embedder.embed([text]))).tolist()

def remember(uid: str, text: str, source: str):
    qdrant.upsert(COLLECTION, points=[PointStruct(
        id=str(uuid.uuid4()),
        vector=embed(text),
        payload={"uid": uid, "text": text, "source": source, "ts": time.time()},
    )])

def recall(uid: str, question: str, k: int = 5) -> list:
    hits = qdrant.query_points(
        COLLECTION,
        query=embed(question),
        query_filter=Filter(must=[FieldCondition(key="uid",
        match=MatchValue(value=uid))]),
        limit=k,
    ).points
    return [h.payload["text"] for h in hits]

# --- Lyzr ---
def ask_lyzr(uid: str, question: str, context: list) -> str:
    ctx = "\n".join("- " + c for c in context) or "none"
    r = requests.post(
        "https://agent-prod.studio.lyzr.ai/v3/inference/chat/",
        headers={"Content-Type": "application/json", "x-api-key": LYZR_API_KEY},
        json={
            "user_id": uid,
            "agent_id": LYZR_AGENT_ID,
            "session_id": f"{LYZR_AGENT_ID}-{uid}",
            "message": f"CONTEXT:\n{ctx}\n\nQUESTION: {question}",
        },
    )
    return r.json()["response"]
```

```

        },
        timeout=60,
    )
    return r.json().get("response", "")

# --- Webhooks from Omi ---
app = FastAPI()

@app.post("/omi/conversation")
def on_conversation(uid: str, payload: Any = Body(...)):
    """Memory creation trigger: fires when a conversation is finished."""
    segments = payload.get("transcript_segments", [])
    transcript = " ".join(s.get("text", "") for s in segments)
    overview = (payload.get("structured") or {}).get("overview", "")
    text = (overview + " " + transcript).strip()
    if text:
        remember(uid, text, source="conversation")
    return {"status": "ok"}

@app.post("/omi/realtime")
def on_realtime(uid: str, session_id: str = "", payload: Any = Body(...)):
    """Real-time transcript: fires with small batches while you speak."""
    segments = payload if isinstance(payload, list) else payload.get("segments", [])
    text = " ".join(s.get("text", "") for s in segments).strip()
    if text:
        remember(uid, text, source="realtime")
    return {"status": "ok"}

# --- Your own question endpoint ---
@app.post("/ask")
def ask(body: dict = Body(...)):
    uid, question = body["uid"], body["question"]
    context = recall(uid, question)
    return {"answer": ask_lyzr(uid, question, context), "context": context}

```

Run it

Terminal 1: backend

```

export LYZR_API_KEY=your_key_here
export LYZR_AGENT_ID=your_agent_id_here
uvicorn app:app --port 8000

```

Terminal 2: public URL

```

ngrok http 8000

```

Copy the HTTPS URL that ngrok prints. In the Omi app's Developer Mode settings, set the memory-creation webhook to `YOUR_URL/omi/conversation` and the real-time transcript webhook to `YOUR_URL/omi/realtime`.

Note: payload shapes can differ slightly between Omi versions. The code reads fields defensively, but check the real JSON you captured on webhook.site in step 3 and adjust the field names if needed.

8. Step 5: Test the full loop

1. Speak a few sentences to Omi, for example: "I need to send the invoice to Rahul by Friday and book the venue on Monday." Wait for the conversation to finish.

2. Open the Qdrant dashboard at <http://localhost:6333/dashboard> and confirm points appeared in the `voice_memory` collection.
3. Ask a question with the command below.

Terminal

```
curl -X POST http://localhost:8000/ask \  
-H "Content-Type: application/json" \  
-d '{"uid": "YOUR_OMI_UID", "question": "What are my action items?"}'
```

If you get a grounded answer back, the loop works. Everything after this is making it a real product.

Troubleshooting

Symptom	Likely cause and fix
Nothing reaches your backend	The tunnel URL changed (free ngrok URLs change on restart). Update the URL in Omi and confirm the path ends in <code>/omi/conversation</code> or <code>/omi/realtime</code> .
Works on <code>webhook.site</code> but not on your server	Your endpoint is slow or returning an error. Return 200 right away and check the unicorn logs.
Qdrant vector size error	The collection size must match the embedding model. <code>bge-small-en-v1.5</code> needs 384. Delete and recreate the collection if you change models.
Lyzr returns an auth error	Check the <code>x-api-key</code> header, and that the <code>agent_id</code> belongs to the same account as the key.
Empty answers	The uid in <code>/ask</code> must match the uid Omi sent, because retrieval is filtered per user.

Making it score well

- **Show the agents working.** Log each step: transcript received, memory stored, memories retrieved, agent called. Observable workflows are part of the score.
- **Use more than one agent.** Split the work across specialized Lyzr agents with a manager, sharing the same Qdrant collection.
- **Store useful metadata.** Add fields like speaker, topic or date to the payload and filter on them. Time-based filters are the base of the habit-tracking track.
- **Give the user control.** A "forget this" endpoint that deletes points is a strong feature for the privacy track.
- **Close the loop.** Send the answer back to the user, for example with Omi's notifications API, so the demo feels like an assistant and not a database.

9. Submission checklist

Submit before 13 October 2026, 11:59 PM IST. The exact submission form and API keys are on the hackathon platform, so check it for the final format.

- ☐ Public, open-source repository with a clean structure and a clear README.

- ☐ All three tools genuinely integrated: Omi voice input, Qdrant retrieval and Lyzr agent reasoning, in one connected loop.
- ☐ Architecture documentation with a diagram (the flow in section 2 is a good starting point).
- ☐ A working demo that someone else can run or watch, showing voice in and an answer out.
- ☐ No API keys committed. Use environment variables and a .env.example file.
- ☐ One build track chosen and named in the README.
- ☐ Solo entry: one person, one submission.

10. Official resources

Lyzr

- [Agent Studio \(sign up\)](#)
- [Quickstart](#)
- [Agent chat API](#)
- [Create agent API](#)
- [Manager Agent](#)
- [Lyzr ADK \(Python\)](#)
- [Lyzr + Qdrant cookbook](#)
- [Full documentation index](#)

Qdrant

- [Local quickstart](#)
- [Documentation home](#)
- [Python client \(qdrant-client\)](#)

Omi

- [Building apps for Omi](#)
- [Integration apps and webhooks](#)
- [Real-time audio streaming](#)
- [Sending notifications](#)
- [Omi on GitHub](#)
- [webhook.site \(inspect payloads\)](#)

Hackathon

- [WhatsApp group \(workshop link and help\)](#)

Submission Guidelines - Stop Prompting, Solo Agents Hackathon

It's time to submit your project for the **Stop Prompting, Solo Agents Hackathon**.

Submit your project:

<https://app.hidevs.xyz/hackathons/stop-prompting-solo-agents-hackathon-2026?tab=submit>

Before submitting, keep these ready:

- **Project Name**
- **Project Title**
- **Build Track**
- **Public GitHub Repository**
- **Deployed Demo URL**
- **Demo Video (Max 5 minutes)**
- **Omi Usage**
- **Qdrant Usage**
- **Lyzr Usage**
- **Project Description**

Deployed Demo URL

Share the **live, working link** of your project. You can deploy using **Vercel, Netlify, Render, or Railway** and submit the deployed URL.

Demo Video - Max 5 Minutes

Share a short video showing your project **in action**. Briefly explain the problem, demonstrate the working project, and highlight how **Omi, Qdrant, and Lyzr** are used.

Build Tracks

Choose the track that best matches your project:

- Meeting & Lecture Intelligence
- Collaborative Multi-Agent Workflows
- Privacy-First Knowledge Systems
- Accessibility & Adaptive Assistants
- Continuous Health & Habit Tracking
- Open Innovation

Final Flow:

Build → Deploy → Push to GitHub → Record Demo → Complete Form → Submit

Hackathon Details:

<https://app.hidevs.xyz/hackathons/stop-prompting-solo-agents-hackathon-2026>

Please test your GitHub, deployment, and demo video links before submitting and make sure all mandatory fields are completed.